

Universidad Tecmilenio

Avance del reto

Programación orientada a objetos

Carrera: Ingeniería en Desarrollo de Software

Alumno: Oscar Gabriel Reyes Delgado

Matricula: T07163611

Profesor: Adalberto Emmanuel Rojas Perea

30-1-2026

Introducción

Para este proyecto de la materia de Programación Orientada a Objetos, me enfoqué en diseñar un sistema que realmente resuelva los problemas de administración de una biblioteca. Como estudiante de Ingeniería en Desarrollo de Software, sé que antes de empezar a programar en Java en NetBeans, lo más importante es tener un buen diseño y saber cómo se van a conectar todas las partes del código.

En este reporte, presento la Fase I del proyecto, donde me encargué de analizar qué es lo que necesita el sistema (como el registro de libros y el control de los préstamos) y cómo estructurarlo usando clases y objetos. Para que todo sea más claro y profesional, utilicé diagramas UML, que me ayudaron a visualizar cómo se relacionan los libros con los usuarios y cómo se deben validar los datos para que el programa no falle.

Mi objetivo con este avance es aplicar los principios de la POO (como la abstracción y la modularidad) para crear una base sólida que sea fácil de escalar y mantener en el futuro.

Requerimientos Funcionales

Los requerimientos funcionales describen las acciones que el sistema debe ser capaz de realizar:

- 1. Gestión de Catálogo:** El sistema permitirá el registro de nuevos ejemplares y la consulta de los existentes. Se almacenará información crítica como título, autor y disponibilidad.
- 2. Gestión de Usuarios:** El sistema debe permitir registrar y consultar la información de los miembros de la biblioteca.
- 3. Gestión de Préstamos:** Debe ser posible registrar un préstamo vinculando a un usuario con un libro, así como registrar la devolución.
- 4. Validación de Datos:** El sistema debe asegurar que los datos ingresados sean correctos y coherentes (por ejemplo, que no existan IDs vacíos o tipos de datos erróneos).
- 5. Control de Inventario y Restricciones:** No se permitirá el préstamo de libros que no tengan ejemplares disponibles en ese momento.
- 6. Generación de Reportes:** El sistema generará impresiones en pantalla que muestren el estado actual de la biblioteca, incluyendo el conteo total de libros, usuarios y préstamos activos.

Requerimientos No Funcionales

Estos requisitos definen las características de calidad del software:

- 1. Usabilidad:** El sistema debe contar con una interfaz clara y ser fácil de operar para el usuario final.
- 2. Robustez:** Debe ser capaz de manejar errores de entrada de forma adecuada sin interrumpir el flujo del programa.
- 3. Eficiencia:** El procesamiento de las consultas y registros debe ser óptimo para garantizar tiempos de respuesta rápidos.
- 4. Escalabilidad:** El diseño basado en clases y objetos debe permitir la adición de nuevas funcionalidades en el futuro (como gestión de multas o categorías) de manera sencilla.

Diseño de clases y objetos

Cada clase ha sido diseñada bajo el principio de **modularidad**, asegurando que cada una tenga una responsabilidad única y clara.

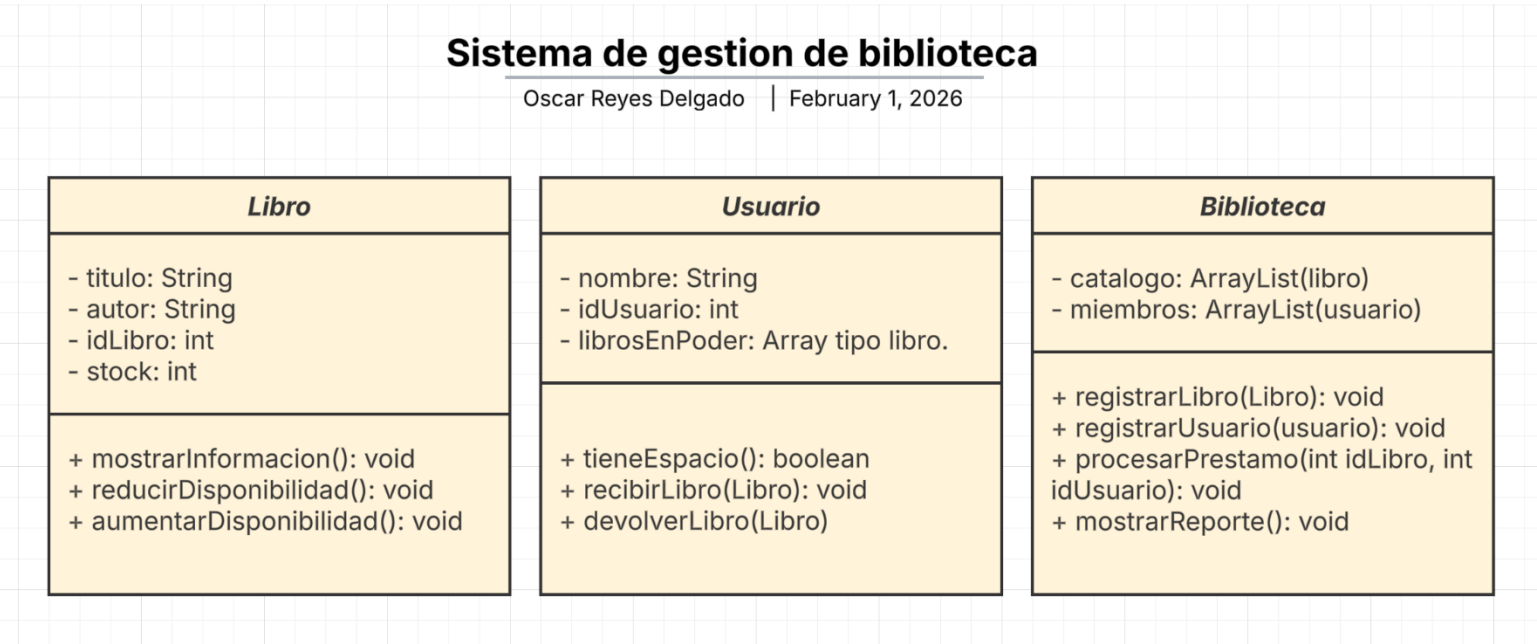


Figura 1.0: Clases graficadas en lenguaje UML.

Clase Libro: La función de esta clase es modelar los datos de los libros físicamente, sus atributos son privados por encapsulamiento, para que cualquier cambio pase por sus métodos.

El método de mostrarInformacion() no retorna ningún valor ya su única misión es imprimir en la consola, reducirDisponibilidad() y aumentarDisponibilidad() son void porque solo modifican el valor de la variable interna stock. Son como el “filtro de seguridad para manejar el inventario.

Clase Usuario: Es la clase que representa al cliente o alumno, los atributos son privados para proteger el nombre y el ID, y la lista de librosEnPoder es privada para que nadie le meta libros al usuario sin que el sistema se entere.

El método tieneEspacio() es boolean porque funciona como una pregunta, devolverá true si tiene espacio y false si esta ocupado. La clase biblioteca necesita esta respuesta lógica para autorizar el préstamo. El método recibirLibro(Libro) es void porque solo toma el objeto libro y lo guarda en la lista del usuario, el parámetro Libro es necesario porque el metodo debe saber exactamente que libro esta recibiendo.

Clase biblioteca: Es el cerebro del programa, los atributos son privados porque las listas de catalogo y miembros son muy importantes, y por eso solamente sus métodos deberían poder añadir o quitar elementos de forma controlada.

El procesarPrestamo(idLibro, idUsuario) es el metodo mas complejo, es void porque ejecuta la transacción y usa el proceso de validación tieneEspacio() de la clase usuario, y si el libro tiene stock, se hace intercambio. Y el mostrarReporte() es void y public porque permite al administrador ver que esta pasando en todo el sistema.

Relación entre clases

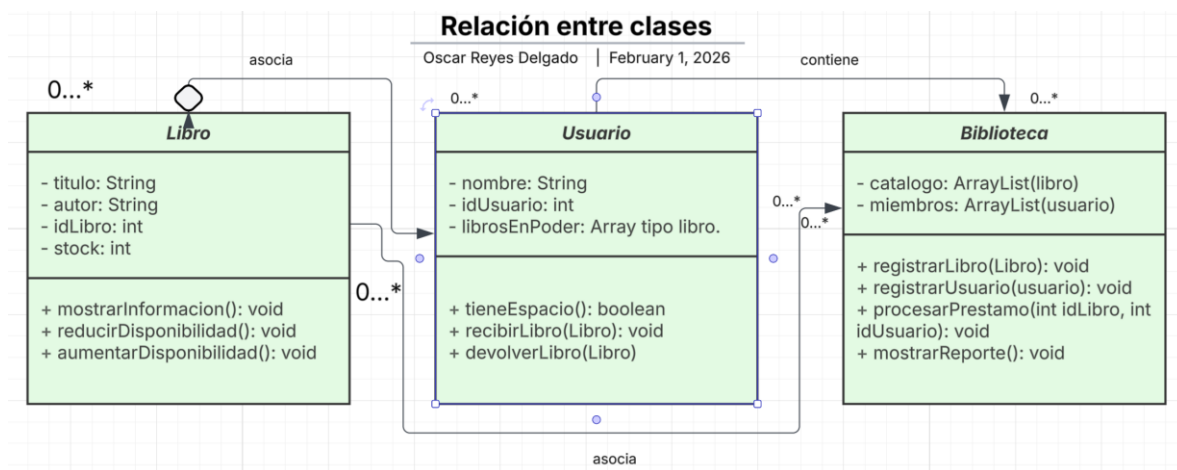


Figura 1.1: Relaciones entre clases

La relación principal se llama **Agregación** y es la que une a la **Biblioteca** con los **Libros** y los **Usuarios**. En el diagrama se ve como un rombo vacío. Esto significa que la Biblioteca "tiene" o "contiene" a los libros y a los miembros, pero no son lo mismo. Es como un equipo de fútbol: el equipo tiene jugadores, pero si el equipo se deshace, los jugadores siguen existiendo por su cuenta. En tu código, esto se traduce en que la Biblioteca tiene listas (ArrayList) para guardar a todos.

Luego está la **Multiplicidad**, que son esos números de que ves en las esquinas. Esto simplemente indica "cuántos" pueden participar en la relación. Se lee como "de cero a

muchos". Esto le dice al programa que tu biblioteca puede empezar totalmente vacía (con cero libros y cero usuarios) y puede crecer hasta tener muchísima gente y muchísimos libros registrados sin que el sistema se trabe, porque las listas son dinámicas.

Finalmente, tenemos la **Asociación** entre el **Usuario** y el **Libro**, que normalmente se marca con una línea o flecha. Esta relación existe porque, cuando se hace un préstamo, el Usuario necesita "conocer" qué libros trae cargando. Por eso, dentro de la clase Usuario, pusimos una lista de libros. Es una relación de uso: el usuario usa el libro por un tiempo, y cuando lo devuelve, esa conexión se rompe y el libro regresa al catálogo general de la biblioteca.

Diagrama de casos de uso

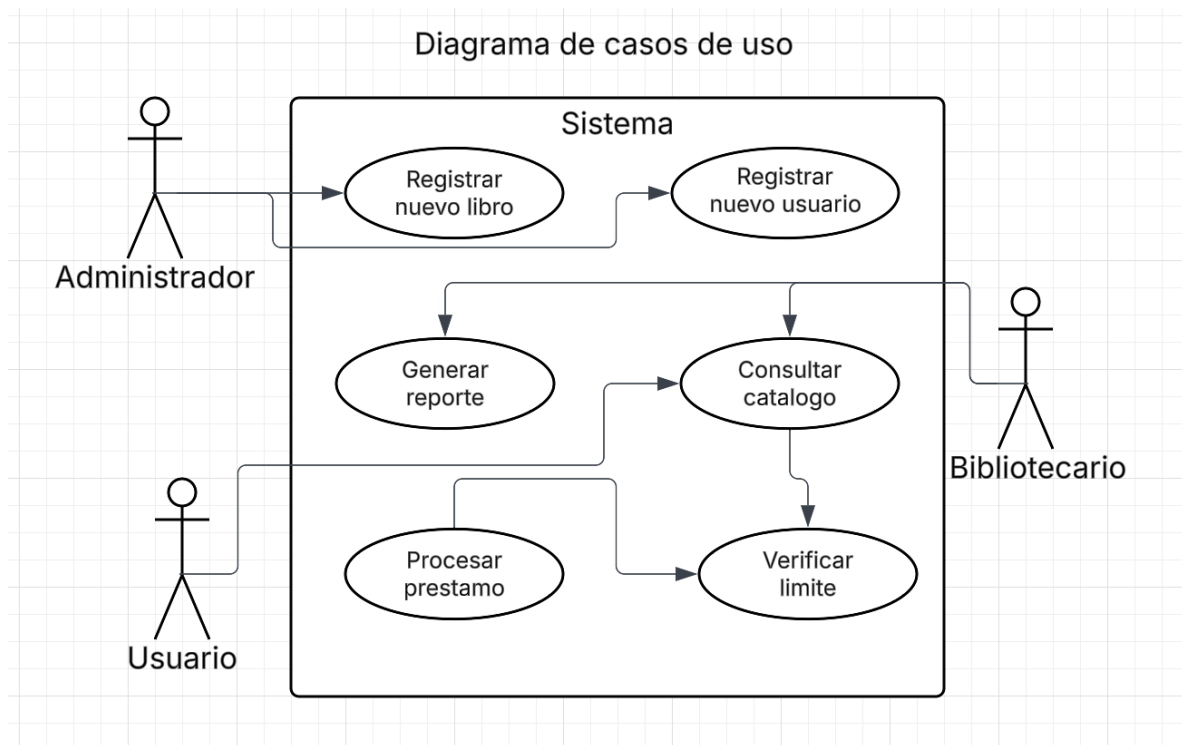


Figura 1.3: Casos de uso

El diagrama de casos de uso representa las interacciones entre los actores (Administrador, Bibliotecario y Usuario) y las funciones del sistema. Se han definido casos de uso clave como el registro de entidades y el procesamiento de préstamos, el cual incluye validaciones automáticas para garantizar que se cumplan las reglas de negocio de la institución, como el límite de dos libros por socio.

- **Administrador:** Es el encargado de alimentar el sistema. Sus funciones principales (casos de uso) son Registrar nuevos libros en el catálogo y Registrar nuevos usuarios en la base de datos de la biblioteca.

- **Bibliotecario:** Es el operador del día a día. Su función principal es Procesar Préstamos, donde el sistema verifica que haya stock y que el usuario no tenga más de dos libros. También se encarga de Generar Reportes para ver cómo va el movimiento de la biblioteca.
- **Usuario (Socio):** Es el beneficiario del sistema. Su interacción principal es Consultar el Catálogo para ver qué libros están disponibles.

Documentación del Diseño

En esta sección se detalla la lógica aplicada para estructurar el sistema, asegurando que los objetos interactúen de forma correcta y segura.

Aplicación del Encapsulamiento

El diseño se basa en el principio de ocultamiento de datos. Todos los atributos de las clases (Libro, Usuario, Biblioteca) se han definido como privados (private). Esto garantiza que el estado de los objetos no pueda ser alterado arbitrariamente desde el exterior. Para interactuar con estos datos, se han creado métodos públicos (public) que actúan como filtros, asegurando que cualquier cambio (como reducir el stock o agregar un libro a un usuario) cumpla con las reglas de validación establecidas.

Lógica de los Métodos y Comportamiento

Se han definido métodos con responsabilidades únicas para mantener la cohesión del sistema:

- **Validaciones Lógicas:** El método `tieneEspacio()` en la clase Usuario devuelve un valor **booleano**, permitiendo que el sistema tome decisiones antes de proceder con un préstamo.
- **Acciones de Estado:** Los métodos de tipo void, como `reducirDisponibilidad()`, se encargan de actualizar la información interna sin necesidad de retornar valores, optimizando la ejecución.
- **Control Centralizado:** El método `procesarPrestamo()` en la clase Biblioteca es el corazón del programa, donde se orquestan las interacciones entre los objetos para cumplir con la regla de máximo dos libros por persona.

Consideración de Requisitos No Funcionales

Para asegurar la calidad del Sistema de Gestión de Biblioteca, se han integrado los siguientes requisitos en el diseño arquitectónico:

- **1. Usabilidad:** El sistema debe contar con una interfaz clara y ser fácil de operar para el usuario final.
 - **En el diseño:** Se logra mediante la clase controladora Biblioteca, la cual centraliza las funciones complejas, permitiendo que el usuario interactúe con

métodos públicos sencillos y directos, ocultando la complejidad del manejo de datos.

- **2. Robustez:** Debe ser capaz de manejar errores de entrada de forma adecuada sin interrumpir el flujo del programa.
 - **En el diseño:** Se implementa a través del **Encapsulamiento**. Al declarar los atributos como `private`, evitamos que se ingresen valores inválidos (como stock negativo). Además, el uso de métodos de validación como `tieneEspacio()` asegura que el sistema se detenga antes de cometer un error lógico.
- **3. Eficiencia:** El procesamiento de las consultas y registros debe ser óptimo para garantizar tiempos de respuesta rápidos.
 - **En el diseño:** Se utilizan estructuras de datos dinámicas (`ArrayList`) en las clases `Usuario` y `Biblioteca`. Esto permite que el acceso a la información de los libros y miembros sea casi instantáneo, optimizando el uso de la memoria RAM del equipo.
- **4. Escalabilidad:** El diseño basado en clases y objetos debe permitir la adición de nuevas funcionalidades en el futuro de manera sencilla.
 - **En el diseño:** El modelo es altamente modular. Gracias a la **Agregación** y la independencia de las clases `Libro` y `Usuario`, es posible añadir módulos de multas, categorías de socios o tipos de libros digitales sin necesidad de modificar el código existente, facilitando el crecimiento del proyecto.

Conclusión

Después de armar todo este diseño, me doy cuenta de que la POO no es solo un concepto teórico, sino una herramienta bien útil para poner orden en problemas de la vida real. Al separar mi lógica en clases como `Libro`, `Usuario` y `Biblioteca`, logré que el sistema sea modular; es decir, que si quiero cambiar algo después, no tenga que romper todo el programa.

Me sirvió mucho aplicar el encapsulamiento para proteger los datos y usar diagramas UML para ver cómo se conectan las partes antes de irme directo a NetBeans. Siento que con esta estructura la implementación en la siguiente fase va a ser mucho más sencilla y el programa va a quedar bien profesional, cumpliendo con todos los requisitos de calidad que me piden.